

Министерство науки и высшего образования Российской
Федерации федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский
университет имени академика С.П. Королева»
Институт информатики и кибернетики

Отчет по лабораторной работе № 6

Дисциплина: «ООП»

Тема «Многопоточное приложение»

Выполнил: Сучков Борис Антонович

Группа: 6201-120303D

Самара 2025

Задание на лабораторную работу

Разработать приложение, формирующее в одном потоке вычислений набор заданий для интегрирования, а во втором потоке – вычисляющее значения интегралов функций.

Official disclaimer (официальное уведомление). Некоторые элементы дизайна, навязанные в задании, являются не очень корректными (например, вместо одного объекта задания, используемого разными нитями, в данном случае разумно использовать очередь заданий, и т.д.). Однако корректная реализация и её исследование по своей сложности выходят за рамки одной лабораторной работы.

Задание 1

Добавьте в класс `Functions` метод, возвращающий значение интеграла функции, вычисленное с помощью численного метода.

В качестве параметров метод должен получать ссылку типа `Function` на объект функции, значения левой и правой границы области интегрирования, а также шаг дискретизации.

Если интервал интегрирования выходит за границы области определения функции, метод должен выбрасывать исключение.

Вычисление значения интеграла должно выполняться по методу трапеций. Для этого вся область интегрирования разбивается на участки, длина которых (кроме одного) равна шагу дискретизации. На каждом таком участке площадь под кривой, описываемой заданной функцией, приближается площадью трапеции, две вершины которой расположены на оси абсцисс на границах участка, а ещё две – на кривой в точках границ участка. Обратите внимание на то, что в область интегрирования необязательно укладывается целое количество шагов дискретизации.

В методе `main()` проверьте работу метода интегрирования. Для этого вычислите интеграл для экспоненты на отрезке от 0 до 1. Определите также, какой шаг дискретизации нужен, чтобы рассчитанное значение отличалось от теоретического в 7 знаке после запятой.

Задание 2

Далее в приложении один поток инструкций будет генерировать задачи для интегрирования, а второй поток – решать их. Для этого потребуется объект задания, через который эти потоки будут взаимодействовать. В объекте будут храниться параметры задания.

Создайте пакет `threads`, в котором будут размещены классы, связанные с потоками.

В пакете `threads` опишите класс `Task`, объект которого должен хранить ссылку на объект интегрируемой функции, границы области интегрирования, шаг дискретизации, а также целочисленное поле, хранящее количество выполняемых заданий. Т.е. объект описывает одно задание.

В главном классе программы опишите метод `nonThread()`, реализующий последовательную (без применения потоков инструкций) версию программы. В методе необходимо создать объект класса `Task` и установить в нём количество выполняемых заданий (минимум 100). После этого в цикле (до количества заданий) нужно выполнить следующие действия:

1. создать и поместить в объект задания объект логарифмической функции, основание которой является случайной величиной, распределённой равномерно на отрезке от 1 до 10;
2. указать в объекте задания левую границу области интегрирования (случайно распределена на отрезке от 0 до 100);
3. указать в объекте задания правую границу области интегрирования (случайно распределена на отрезке от 100 до 200);
4. указать в объекте задания шаг дискретизации (случайно распределён на отрезке от 0 до 1);
5. вывести в консоль сообщение вида "Source <левая граница> <правая граница> <шаг дискретизации>";
6. вычислить значение интеграла для параметров из объекта задания;
7. вывести в консоль сообщение вида "Result <левая граница> <правая граница> <шаг дискретизации> <результат интегрирования>".

Проверьте работу метода, вызвав его в методе `main()`.

Задание 3

В пакете `threads` создайте два следующих класса. При их реализации воспользуйтесь фрагментами последовательной версии программы из предыдущего задания.

Класс `SimpleGenerator` должен реализовывать интерфейс `Runnable`, получать в конструкторе и сохранять в своё поле ссылку на объект типа `Task`, а в методе `run()` в цикле должны формироваться задачи и заноситься в полученный объект задания, а также выводиться сообщения в консоль.

Класс `SimpleIntegrator` должен реализовывать интерфейс `Runnable`, получать в конструкторе и сохранять в своё поле ссылку на объект типа `Task`, а в методе `run()` в цикле должны решаться задачи, данные для которых берутся из полученного объекта задания, а также выводиться сообщения в консоль.

В главном классе программы создайте метод `simpleThreads()`. В нём создайте объект задания, укажите количество выполняемых заданий (минимум 100), создайте и запустите два потока вычислений, основанных на описанных классах `SimpleGenerator` и `SimpleIntegrator`. Проверьте работу метода, вызвав его в методе `main()`.

Попробуйте запускать программу (несколько раз). Попробуйте запускать программу, изменяя приоритеты потоков перед их запуском. Убедитесь, что в некоторых случаях интегрирующий поток может прекращать своё выполнение из-за исключения `NullPointerException`. Определите причину возникновения этого исключения и сделайте так, чтобы оно не возникало (используйте какое-нибудь простое решение, без синхронизации и нотификаций). Попробуйте запускать программу (несколько раз). Убедитесь, что в некоторых случаях интегрирующий поток выводит сообщение с набором данных, который не встречается в сообщениях генерирующего потока (например, левая граница области интегрирования оказывается взята из одного задания, а правая граница и шаг дискретизации – из другого). Определите причину возникновения таких ситуаций и устраните её, используя блоки синхронизации в методах `run()` генерирующего и интегрирующего потоков. Убедитесь, что в коде нигде не возникает необоснованно длительной блокировки объектов.

Задание 4

Определите причину того, что не все сгенерированные задания оказываются выполнены интегрирующим потоком.

Для устранения этой проблемы потребуется дополнительный объект, представляющий собой одноместный семафор, различающий операции чтения и записи в защищаемый объект (пример класса таких объектов был приведён в лекции).

В пакете `threads` создайте два следующих класса. При их реализации воспользуйтесь фрагментами классов из предыдущего задания.

Класс `Generator` должен расширять класс `Thread`, получать в конструкторе и сохранять в свои поля ссылки на объект типа `Task` и на объект семафора, а в методе `run()` должны выполняться те же действия, что и в предыдущей версии генерирующего класса.

Класс `Integrator` должен расширять класс `Thread`, получать в конструкторе и сохранять в свои поля ссылки на объект типа `Task` и на объект семафора, а в методе `run()` должны выполняться те же действия, что и в предыдущей версии интегрирующего класса.

Отличие этих классов от предыдущих версий должно заключаться в том, что вместо средств синхронизации в методах `run()` должны использоваться возможности семафора.

В главном классе программы создайте метод `complicatedThreads()`. В нём создайте объект задания, укажите количество выполняемых заданий (минимум 100), создайте и запустите два потока вычислений классов `Generator` и `Integrator`. Проверьте работу метода, вызвав его в методе `main()`.

Попробуйте запускать программу (несколько раз). Попробуйте запускать программу, изменяя приоритеты потоков перед их запуском. Сделайте так, чтобы после создания ваших потоков основной поток программы выжидал 50 миллисекунд, после чего

прерывал работу ваших потоков путём вызова метода `interrupt()`. Измените код ваших классов потоков так, чтобы прерывание происходило и происходило корректно.

Задание 1

Я добавил в класс `Functions` метод, возвращающий значение интеграла функции, вычисленное с помощью численного метода.

В качестве параметров метод получает ссылку типа `Function` на объект функции, значения левой и правой границы области интегрирования, а также шаг дискретизации.

Если интервал интегрирования выходит за границы области определения функции, метод выбрасывает исключение.

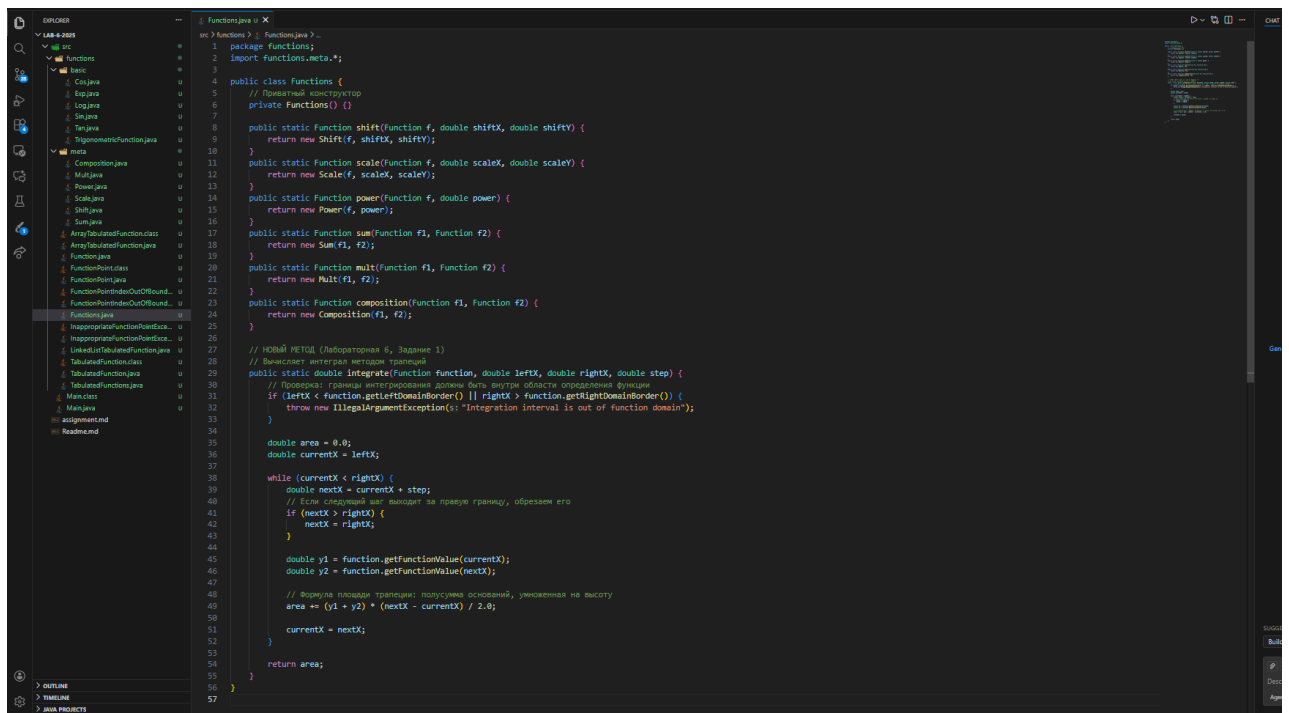
Вычисление значения интеграла выполняется по методу трапеций. Для этого вся область интегрирования разбивается на участки, длина которых (кроме одного) равна шагу дискретизации. На каждом таком участке площадь под кривой, описываемой заданной функцией, приближается площадью трапеции, две вершины которой расположены на оси абсцисс на границах участка, а ещё две – на кривой в точках границ участка. В область интегрирования необязательно укладывается целое количество шагов дискретизации.

В методе `main()` я проверил работу метода интегрирования. Для этого я вычислил интеграл для экспоненты на отрезке от 0 до 1. Определил также, какой шаг дискретизации нужен, чтобы рассчитанное значение отличалось от теоретического в 7 знаке после запятой.

Чтобы достичь точности 10^{-7} (седьмой знак после запятой) при интегрировании экспоненты на отрезке $[0, 1]$ методом трапеций, шаг дискретизации должен быть равен 0.000390625.

Этот результат получается путем последовательного деления начального шага 0.1 на 2^8 раз:

$0.1 / 2^8 = 0.000390625$. При этом шаге погрешность составляет примерно $3.4 \cdot 10^{-8}$, что удовлетворяет условию задачи (меньше 10^{-7}).



Задание 2

Далее в приложении один поток инструкций генерирует задачи для интегрирования, а второй поток – решает их. Для этого требуется объект задания, через который эти потоки будут взаимодействовать. В объекте хранятся параметры задания.

Я создайте пакет `threads`, в котором размещены классы, связанные с потоками.

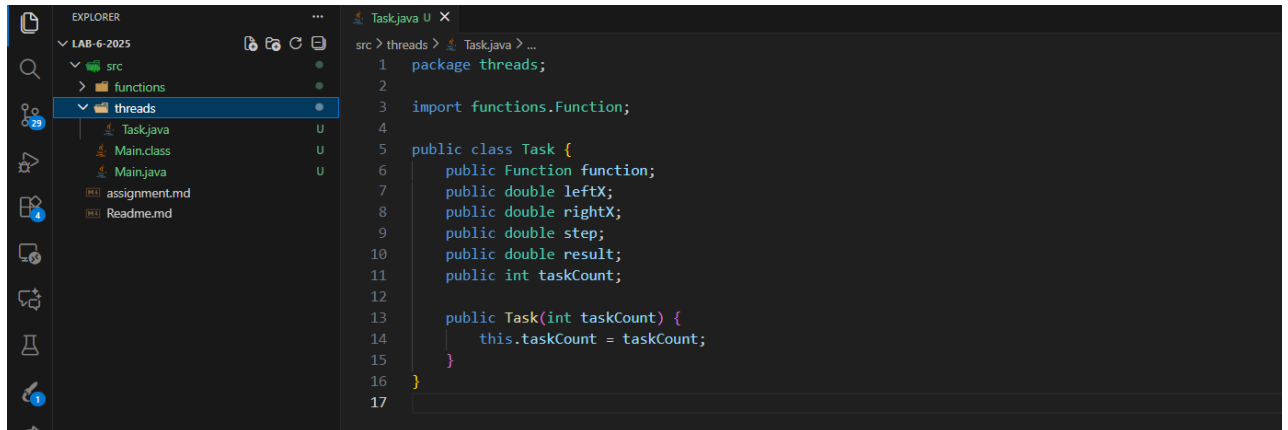
В пакете `threads` я описал класс `Task`, объект которого хранит ссылку на объект интегрируемой функции, границы области интегрирования, шаг дискретизации, а также целочисленное поле, хранящее количество выполняемых заданий. Т.е. объект описывает одно задание.

В главном классе программы я описал метод `nonThread()`, реализующий последовательную (без применения потоков инструкций) версию программы. В методе я создал объект класса `Task` и установил в нём количество выполняемых заданий (минимум 100). После этого в цикле (до количества заданий) я выполнил следующие действия:

1. Я создал и поместил в объект задания объект логарифмической функции, основание которой является случайной величиной, распределённой равномерно на отрезке от 1 до 10;
2. Указал в объекте задания левую границу области интегрирования (случайно распределена на отрезке от 0 до 100);
3. Указал в объекте задания правую границу области интегрирования (случайно распределена на отрезке от 100 до 200);
4. Указал в объекте задания шаг дискретизации (случайно распределён на отрезке от 0 до 1);

5. Вывел в консоль сообщение вида "Source <левая граница> <правая граница> <шаг дискретизации>";
6. Вычислил значение интеграла для параметров из объекта задания;
7. Вывел в консоль сообщение вида "Result <левая граница> <правая граница> <шаг дискретизации> <результат интегрирования>".

Я проверьте работу метода, вызвав его в методе main().



Задание 3

В пакете threads я создал два следующих класса. При их реализации я воспользовался фрагментами последовательной версии программы из предыдущего задания.

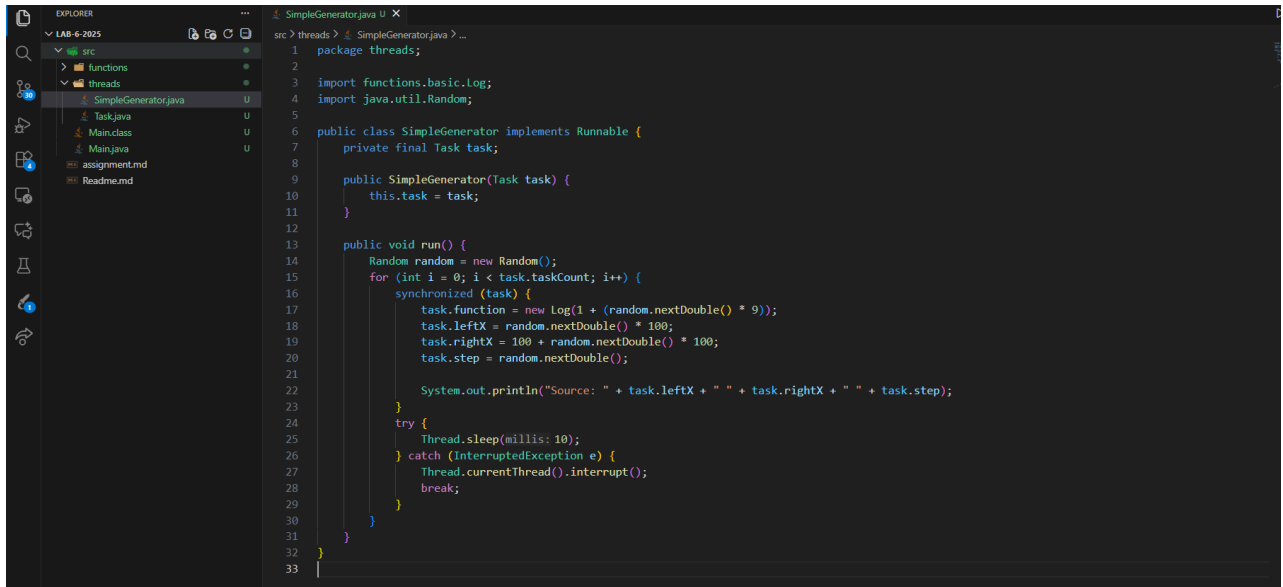
Класс SimpleGenerator реализует интерфейс Runnable, получает в конструкторе и сохраняет в своё поле ссылку на объект типа Task, а в методе run() в цикле формируются задачи и заносятся в полученный объект задания, а также выводятся сообщения в консоль.

Класс SimpleIntegrator реализует интерфейс Runnable, получает в конструкторе и сохраняет в своё поле ссылку на объект типа Task, а в методе run() в цикле решаются задачи, данные для которых берутся из полученного объекта задания, а также выводятся сообщения в консоль.

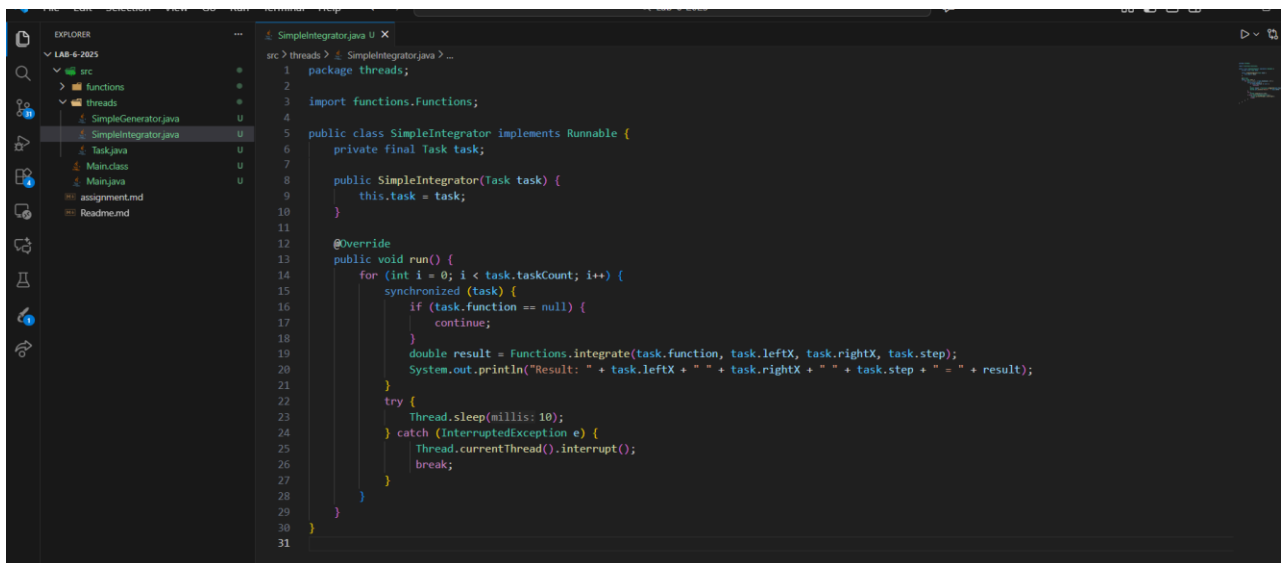
В главном классе программы я создал метод simpleThreads(). В нём создал объект задания, указал количество выполняемых заданий (минимум 100), создал и запустил два потока вычислений, основанных на описанных классах SimpleGenerator и SimpleIntegrator. Я проверил работу метода, вызвав его в методе main().

Я попробовал запускать программу (несколько раз). Попробовал запускать программу, изменяя приоритеты потоков перед их запуском. Убедился, что в некоторых случаях интегрирующий поток может прекращать своё выполнение из-за исключения NullPointerException. Определил причину возникновения этого исключения и сделал так, чтобы оно не возникало (использовал простое решение, без синхронизации и нотификаций). Я попробовал запускать программу (несколько раз). Убедился, что в некоторых случаях интегрирующий поток выводит сообщение с

набором данных, который не встречается в сообщениях генерирующего потока (например, левая граница области интегрирования оказывается взята из одного задания, а правая граница и шаг дискретизации – из другого). Определил причину возникновения таких ситуаций и устранил её, используя блоки синхронизации в методах `run()` генерирующего и интегрирующего потоков. Убедился, что в коде нигде не возникает необоснованно длительной блокировки объектов.



```
1 package threads;
2
3 import functions.basic.log;
4 import java.util.Random;
5
6 public class SimpleGenerator implements Runnable {
7     private final Task task;
8
9     public SimpleGenerator(Task task) {
10         this.task = task;
11     }
12
13     public void run() {
14         Random random = new Random();
15         for (int i = 0; i < task.taskCount; i++) {
16             synchronized (task) {
17                 task.function = new Log(1 + (random.nextDouble() * 9));
18                 task.leftX = random.nextDouble() * 100;
19                 task.rightX = 100 + random.nextDouble() * 100;
20                 task.step = random.nextDouble();
21
22                 System.out.println("Source: " + task.leftX + " " + task.rightX + " " + task.step);
23             }
24             try {
25                 Thread.sleep(10);
26             } catch (InterruptedException e) {
27                 Thread.currentThread().interrupt();
28                 break;
29             }
30         }
31     }
32 }
33
```



```
1 package threads;
2
3 import functions.Functions;
4
5 public class SimpleIntegrator implements Runnable {
6     private final Task task;
7
8     public SimpleIntegrator(Task task) {
9         this.task = task;
10     }
11
12     @Override
13     public void run() {
14         for (int i = 0; i < task.taskCount; i++) {
15             synchronized (task) {
16                 if (task.function == null) {
17                     continue;
18                 }
19                 double result = Functions.integrate(task.function, task.leftX, task.rightX, task.step);
20                 System.out.println("Result: " + task.leftX + " " + task.rightX + " " + task.step + " " + result);
21             }
22             try {
23                 Thread.sleep(10);
24             } catch (InterruptedException e) {
25                 Thread.currentThread().interrupt();
26                 break;
27             }
28         }
29     }
30 }
31
```

Задание 4

Я определите причину того, что не все сгенерированные задания оказываются выполнены интегрирующим потоком.

Для устранения этой проблемы потребуется дополнительный объект, представляющий собой одноместный семафор, различающий операции чтения и записи в защищаемый объект (пример класса таких объектов был приведён в лекции).

В пакете threads я создал два следующих класса. При их реализации я воспользовался фрагментами классов из предыдущего задания.

Класс Generator расширяет класс Thread, получает в конструкторе и сохраняет в свои поля ссылки на объект типа Task и на объект семафора, а в методе run() выполняются те же действия, что и в предыдущей версии генерирующего класса.

Класс Integrator расширяет класс Thread, получают в конструкторе и сохраняют в свои поля ссылки на объект типа Task и на объект семафора, а в методе run() выполняются те же действия, что и в предыдущей версии интегрирующего класса.

Отличие этих классов от предыдущих версий заключается в том, что вместо средств синхронизации в методах run() используются возможности семафора.

В главном классе программы я создал метод complicatedThreads(). В нём я создал объект задания, указал количество выполняемых заданий (минимум 100), создал и запустил два потока вычислений классов Generator и Integrator. Я проверил работу метода, вызвав его в методе main().

Я попробовал запускать программу (несколько раз). Попробовал запускать программу, изменяя приоритеты потоков перед их запуском. Сделал так, чтобы после создания ваших потоков основной поток программы выжидал 50 миллисекунд, после чего прерывал работу моих потоков путём вызова метода interrupt(). Изменил код ваших классов потоков так, чтобы прерывание происходило и происходило корректно.

```
1 package threads;
2
3 public class SimpleSemaphore {
4     private boolean canWrite = true;
5
6     public synchronized void beginWrite() throws InterruptedException {
7         while (!canWrite) {
8             wait();
9         }
10    }
11
12    public synchronized void endWrite() {
13        canWrite = false;
14        notifyAll();
15    }
16
17    public synchronized void beginRead() throws InterruptedException {
18        while (canWrite) {
19            wait();
20        }
21    }
22
23    public synchronized void endRead() {
24        canWrite = true;
25        notifyAll();
26    }
27 }
28
```

```
6 public class Generator extends Thread {
7     private final Task task;
8     private final SimpleSemaphore semaphore;
9
10    public Generator(Task task, SimpleSemaphore semaphore) {
11        this.task = task;
12        this.semaphore = semaphore;
13    }
14
15    @Override
16    public void run() {
17        Random random = new Random();
18        while (task.taskCount > 0 && !isInterrupted()) {
19            try {
20                semaphore.beginWrite();
21
22                task.function = new Log(1 + (random.nextDouble() * 9));
23                task.leftX = random.nextDouble() * 100;
24                task.rightX = 100 + random.nextDouble() * 100;
25                task.step = random.nextDouble();
26
27                task.taskCount--;
28
29                System.out.println("Source: " + task.leftX + " " + task.rightX + " " + task.step);
30
31                semaphore.endWrite();
32            } catch (InterruptedException e) {
33                interrupt();
34            }
35        }
36    }
37 }
38
39
```

```
3 src > threads > Integrator.java > ...
4 import functions.Functions;
5
6 public class Integrator extends Thread {
7     private final Task task;
8     private final SimpleSemaphore semaphore;
9
10    public Integrator(Task task, SimpleSemaphore semaphore) {
11        this.task = task;
12        this.semaphore = semaphore;
13    }
14
15    public void run() {
16        while (!isInterrupted() && task.taskCount >= 0) {
17            try {
18                semaphore.beginRead();
19
20                if (task.function == null) {
21                    semaphore.endRead();
22                    break;
23                }
24
25                double result = Functions.integrate(task.function, task.leftX, task.rightX, task.step);
26                System.out.println("Result: " + task.leftX + " " + task.rightX + " " + task.step + " = " + result);
27
28                semaphore.endRead();
29            } catch (InterruptedException e) {
30                interrupt();
31            }
32        }
33    }
34 }
35
```

```

1 import functions.*;
2 import functions.basic.Exp;
3 import functions.basic.Log;
4 import threads.*;
5 import java.util.Random;
6
7 public class Main {
8     // --- Main ---
9     public static void main(String[] args) {
10         // --- Example 1: Metropolis ---
11         System.out.println("Example 1: Metropolis ---");
12         Function exp = new Exp();
13         double res = Functions.integrate(exp, leftX: 0, rightX: 1, step: 0.1);
14         double theoretical = Math.E - 1;
15         System.out.println("Metropolis (var 0.1): " + res);
16         System.out.println("Theoretical: " + theoretical);
17
18         double step = 0.1;
19         while (Math.abs(Functions.integrate(exp, leftX: 0, rightX: 1, step) - theoretical) >= 1e-7) {
20             step /= 2;
21         }
22         System.out.println("Var size towards 1e-7: " + step);
23         System.out.println("Metropolis || arm error: " + Functions.integrate(exp, leftX: 0, rightX: 1, step));
24
25         // --- Example 2: NonThread ---
26         System.out.println("Example 2: NonThread ---");
27         nonThread();
28
29         // --- Example 3: Simple Threads ---
30         System.out.println("Example 3: Simple Threads ---");
31         simpleThreads();
32
33         // --- Example 4: Complicated Threads (Semaphore) ---
34         System.out.println("Example 4: Complicated Threads ---");
35         complicatedThreads();
36
37     }
38
39     public static void nonThread() {
40         Task task = new Task(taskCount: 100);
41         Random random = new Random();
42         for (int i = 0; i < task.taskCount; i++) {
43             task.function = new Log(1 + (random.nextDouble() * 9));
44             task.leftX = random.nextDouble() * 100;
45             task.rightX = 100 + random.nextDouble() * 100;
46             task.step = random.nextDouble();
47
48             System.out.println("Source: " + task.leftX + " " + task.rightX + " " + task.step);
49
50             double r = Functions.integrate(task.function, task.leftX, task.rightX, task.step);
51             System.out.println("Result: " + task.leftX + " " + task.rightX + " " + task.step + " " + r);
52         }
53     }
54
55     public static void simpleThreads() {
56         Task task = new Task(taskCount: 100);
57         SimpleGenerator generator = new SimpleGenerator(task);
58
59         Thread t1 = new Thread(generator);
60         Thread t2 = new Thread(generator);
61
62         t1.start();
63         t2.start();
64
65         try {
66             t1.join();
67             t2.join();
68         } catch (InterruptedException e) {
69             e.printStackTrace();
70         }
71     }
72
73     public static void complicatedThreads() {
74         Task task = new Task(taskCount: 100);
75         SimpleSemaphore semaphore = new SimpleSemaphore();
76
77         Generator generator = new Generator(task, semaphore);
78         Integrator integrator = new Integrator(task, semaphore);
79
80         generator.start();
81         integrator.start();
82
83         try {
84             Thread.sleep(500);
85             generator.interrupt();
86             integrator.interrupt();
87         } catch (InterruptedException e) {
88             e.printStackTrace();
89         }
90     }
91 }

```

```

1 public class Main {
2     // --- Main ---
3     public static void main(String[] args) {
4         // --- Example 1: Metropolis ---
5         System.out.println("Example 1: Metropolis ---");
6         Function exp = new Exp();
7         double res = Functions.integrate(exp, leftX: 0, rightX: 1, step: 0.1);
8         double theoretical = Math.E - 1;
9         System.out.println("Metropolis (var 0.1): " + res);
10        System.out.println("Theoretical: " + theoretical);
11
12        double step = 0.1;
13        while (Math.abs(Functions.integrate(exp, leftX: 0, rightX: 1, step) - theoretical) >= 1e-7) {
14            step /= 2;
15        }
16        System.out.println("Var size towards 1e-7: " + step);
17        System.out.println("Metropolis || arm error: " + Functions.integrate(exp, leftX: 0, rightX: 1, step));
18
19        // --- Example 2: NonThread ---
20        System.out.println("Example 2: NonThread ---");
21        nonThread();
22
23        // --- Example 3: Simple Threads ---
24        System.out.println("Example 3: Simple Threads ---");
25        simpleThreads();
26
27        // --- Example 4: Complicated Threads (Semaphore) ---
28        System.out.println("Example 4: Complicated Threads ---");
29        complicatedThreads();
30
31    }
32
33    public static void nonThread() {
34        Task task = new Task(taskCount: 100);
35        Random random = new Random();
36        for (int i = 0; i < task.taskCount; i++) {
37            task.function = new Log(1 + (random.nextDouble() * 9));
38            task.leftX = random.nextDouble() * 100;
39            task.rightX = 100 + random.nextDouble() * 100;
40            task.step = random.nextDouble();
41
42            System.out.println("Source: " + task.leftX + " " + task.rightX + " " + task.step);
43
44            double r = Functions.integrate(task.function, task.leftX, task.rightX, task.step);
45            System.out.println("Result: " + task.leftX + " " + task.rightX + " " + task.step + " " + r);
46        }
47    }
48
49    public static void simpleThreads() {
50        Task task = new Task(taskCount: 100);
51        SimpleGenerator generator = new SimpleGenerator(task);
52
53        Thread t1 = new Thread(generator);
54        Thread t2 = new Thread(generator);
55
56        t1.start();
57        t2.start();
58
59        try {
60            t1.join();
61            t2.join();
62        } catch (InterruptedException e) {
63            e.printStackTrace();
64        }
65    }
66
67    public static void complicatedThreads() {
68        Task task = new Task(taskCount: 100);
69        SimpleSemaphore semaphore = new SimpleSemaphore();
70
71        Generator generator = new Generator(task, semaphore);
72        Integrator integrator = new Integrator(task, semaphore);
73
74        generator.start();
75        integrator.start();
76
77        try {
78            Thread.sleep(500);
79            generator.interrupt();
80            integrator.interrupt();
81        } catch (InterruptedException e) {
82            e.printStackTrace();
83        }
84    }
85 }

```

```
PS C:\projects\Lab-6-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49780' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Borus\AppData\Roaming\Code\User\workspaceStorage\202763ced4c9ee57ffc339070f0064\redhat.java\jdt_ws\Lab-6-2025_4789df95\bin' 'Main'

--- Задание 1: Интегрирование экспоненты ---
Результат (шаг 0.1): 1.7191734913893144
Точное значение: 1.718281828459045
Шаг для точности 1e-7: 7.8125E-4
Результат с этим шагом: 1.7182819158553764

--- Задание 2: nonThread() ---
Source: 90.83524560742258 177.3597287968371 0.7635973923795976
Result: 90.83524560742258 177.3597287968371 0.7635973923795976 219.7463196066104
Source: 40.7982672273436 127.60893760289521 0.935913925734002
Result: 40.7982672273436 127.60893760289521 0.935913925734002 172.53858323302006
Source: 56.118550458855545 115.68523621789998 0.5473689940690507
Result: 56.118550458855545 115.68523621789998 0.5473689940690507 147.92536812463757
Source: 70.02906072636446 128.2634021726611 0.30966597913137506
Result: 70.02906072636446 128.2634021726611 0.30966597913137506 121.44218611403458
Source: 30.211624521005486 144.74861538770023 0.049160134422789104
Result: 30.211624521005486 144.74861538770023 0.049160134422789104 356.9008288659677
Source: 77.01481416855688 106.70558224966351 0.6808942167966818
Result: 77.01481416855688 106.70558224966351 0.6808942167966818 205.3671799415285
Source: 0.4004234319958999 198.95465493931476 0.1072646222871898
Result: 0.4004234319958999 198.95465493931476 0.1072646222871898 1302.7360786809684
Source: 44.32274109259755 139.0908669368903 0.5882227749920578
Result: 44.32274109259755 139.0908669368903 0.5882227749920578 468.312689542289
Source: 62.498765596660164 123.84207782428079 0.8730822269811649
Result: 62.498765596660164 123.84207782428079 0.8730822269811649 134.71262342633628
Source: 67.90637334825902 160.46748655618507 0.658414252393193
Result: 67.90637334825902 160.46748655618507 0.658414252393193 264.0324670291568
Source: 59.85142167477022 119.30797051625869 0.7421804103380338
Result: 59.85142167477022 119.30797051625869 0.7421804103380338 161.51212235351656
Source: 57.073597962808456 179.63486658827398 0.5811631716488463
Result: 57.073597962808456 179.63486658827398 0.5811631716488463 352.83808163203975
Source: 13.114788668701749 105.10394649697093 0.8269210355388202
Result: 13.114788668701749 105.10394649697093 0.8269210355388202 225.4126401103375
Source: 41.05262276307936 177.40650400494363 0.8149074779126326
Result: 41.05262276307936 177.40650400494363 0.8149074779126326 623.1073457349449
Source: 28.70576884332848 137.11964828502775 0.9120582075502253
Result: 28.70576884332848 137.11964828502775 0.9120582075502253 207.43570200852287
Source: 82.546442361553 129.20300844525070 0.09006431299658060
Result: 82.546442361553 129.20300844525070 0.09006431299658060 95.2505852582973
Source: 98.80950984625409 127.96925075530731 0.08471362726916254
Result: 98.80950984625409 127.96925075530731 0.08471362726916254 86.40899629403955
Source: 23.666479380666615 118.62095551925027 0.026096441631766387
Result: 23.666479380666615 118.62095551925027 0.026096441631766387 309.5009255740988
Source: 30.836574852167985 150.39654542258518 0.48013559182388543
Result: 30.836574852167985 150.39654542258518 0.48013559182388543 233.48261186938973
Source: 14.635973080395637 131.10312986543525 0.4288154567266478
Result: 14.635973080395637 131.10312986543525 0.4288154567266478 479.8741641328055
Source: 69.22023113693034 127.01426769202277 0.8346479411657236
Result: 69.22023113693034 127.01426769202277 0.8346479411657236
```

```
PS C:\projects\Lab-6-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49780' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Borus\AppData\Roaming\Code\User\workspaceStorage\202763ced4c9ee57ffc339070f0064\redhat.java\jdt_ws\Lab-6-2025_4789df95\bin' 'Main'

Result: 14.635973080395637 131.10312986543525 0.4288154567266478 479.8741641328055
Source: 69.22023113693034 127.01426769202277 0.8346479411657236
Result: 69.22023113693034 127.01426769202277 0.8346479411657236 145.90573008530424
Source: 0.6537957424203267 146.87592083631688 0.08432801344876217
Result: 0.6537957424203267 146.87592083631688 0.08432801344876217 257.6453884896245
Source: 97.4444326106949 134.89327133712558 0.7871951951831303
Result: 97.4444326106949 134.89327133712558 0.7871951951831303 217.27070402226246
Source: 67.30927480454481 171.0188211226844 0.11603143508713698
Result: 67.30927480454481 171.0188211226844 0.11603143508713698 242.1655411323633
Source: 33.19314522759801 178.5171846406326 0.12089014420275268
Result: 33.19314522759801 178.5171846406326 0.12089014420275268 320.0481144957109
Source: 69.93867265462956 168.57360150541865 0.22106484366341317
Result: 69.93867265462956 168.57360150541865 0.22106484366341317 554.490598068943
Source: 54.511643700444424 109.61816661505091 0.38597015946671087
Result: 54.511643700444424 109.61816661505091 0.38597015946671087 108.09992013175568
Source: 97.31394928004061 105.0755573398775 0.352621358075179
Result: 97.31394928004061 105.0755573398775 0.352621358075179 262.43004438232356
Source: 19.870375538796214 197.40939550312282 0.5955683150866211
Result: 19.870375538796214 197.40939550312282 0.5955683150866211 382.98049153689567
Source: 45.054405104353854 138.61568662771353 0.6645329947486254
Result: 45.054405104353854 138.61568662771353 0.6645329947486254 263.2436901279662
Source: 70.02206777755318 177.72990722988112 0.935705953142
Result: 70.02206777755318 177.72990722988112 0.935705953142 273.4768522105292
Source: 99.15187434907159 114.44746257561926 0.7845183241925262
Result: 99.15187434907159 114.44746257561926 0.7845183241925262 106.86323004064192
Source: 24.542638886818057 134.61741675163788 0.6809123289381692
Result: 24.542638886818057 134.61741675163788 0.6809123289381692 215.2102302483263
Source: 55.00004724394384 178.15451771126754 0.7325260068834915
Result: 55.00004724394384 178.15451771126754 0.7325260068834915 467.589996166708
Source: 10.946599425774107 192.82474220875752 0.7568783480709061
Result: 10.946599425774107 192.82474220875752 0.7568783480709061 403.5967497992543
Source: 22.318580247655206 148.39470680972238 0.050553826847167915
Result: 22.318580247655206 148.39470680972238 0.050553826847167915 429.6291802306224
Source: 89.26752404793551 124.79544471085046 0.6415499354554143
Result: 89.26752404793551 124.79544471085046 0.6415499354554143 77.59440115168141
Source: 91.36260463983956 155.23624434680764 0.9603650321694995
Result: 91.36260463983956 155.23624434680764 0.9603650321694995 165.59589941807226
Source: 59.994253833582434 100.61434612994834 0.5460677573942132
Result: 59.994253833582434 100.61434612994834 0.5460677573942132 80.53929501516176
Source: 60.80825762870721 155.10149782950847 0.6155037300054959
Result: 60.80825762870721 155.10149782950847 0.6155037300054959 192.58726320354236
Source: 68.25341528203143 134.44850420212472 0.06695927642577959
Result: 68.25341528203143 134.44850420212472 0.06695927642577959 478.6118147971285
Source: 52.704857510937295 178.33405597891263 0.7640153132013618
Result: 52.704857510937295 178.33405597891263 0.7640153132013618 326.38791926763537
Source: 42.128911214262686 105.99429911776257 0.7509896439617003
Result: 42.128911214262686 105.99429911776257 0.7509896439617003 204.32880921300347
Source: 64.8843790656799 100.40474028755976 0.3882790885020698
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\projects\Lab-6-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49780' '-XX:+ShowCodeDetail
sInExceptionMessages' '-cp' 'C:\Users\Borus\AppData\Roaming\Code\User\workspaceStorage\202763ceddce9ee57ffc339070f0064\redhat.java\jdt_ws\Lab-6-2025_4789df95\bin' 'Main'
Source: 64.8843790656799 100.40474028755976 0.3882790885020698
Result: 64.8843790656799 100.40474028755976 0.3882790885020698 126.4609271181
Source: 28.009010896748265 169.44625634975526 0.26407738810242765
Result: 28.009010896748265 169.44625634975526 0.26407738810242765 491.174018594771
Source: 36.218794839957 152.77213198804805 0.13835881972794584
Result: 36.218794839957 152.77213198804805 0.13835881972794584 240.1214401448262
Source: 10.851641737880346 115.50319954957878 0.10568139418881639
Result: 10.851641737880346 115.50319954957878 0.10568139418881639 280.38787699443145
Source: 25.11596756457104 114.41328817017266 0.625277633186036
Result: 25.11596756457104 114.41328817017266 0.625277633186036 172.5452755965849
Source: 32.258506142553436 128.92741265016028 0.060065232337082164
Result: 32.258506142553436 128.92741265016028 0.060065232337082164 263.54984971610713
Source: 26.64468394472643 150.04022131861944 0.890623363778274
Result: 26.64468394472643 150.04022131861944 0.890623363778274 1028.426733190586
Source: 44.24869814693142 185.17238488877346 0.8258452624591345
Result: 44.24869814693142 185.17238488877346 0.8258452624591345 115.77950548845209
Source: 78.6794636554664 186.01783016459143 0.6188909455344053
Result: 78.6794636554664 186.01783016459143 0.6188909455344053 306.8779569901498
Source: 61.62236369774092 144.5947843612929 0.5152286976757928
Result: 61.62236369774092 144.5947843612929 0.5152286976757928 415.80674735635915
Source: 32.455768803859456 139.9988360068246 0.9089676344590494
Result: 32.455768803859456 139.9988360068246 0.9089676344590494 250.71058715584067
Source: 0.8051067652700916 163.76630013120524 0.9643477064475541
Result: 0.8051067652700916 163.76630013120524 0.9643477064475541 3339.721733851736
Source: 87.92623800228291 145.29141165031137 0.6711677344760797
Result: 87.92623800228291 145.29141165031137 0.6711677344760797 152.33330168069925
Source: 18.063724642491486 148.40094262535487 0.0460961441122493
Result: 18.063724642491486 148.40094262535487 0.0460961441122493 440.0582806973229
Source: 19.30857851073683 135.97508170510497 0.966227559207326
Result: 19.30857851073683 135.97508170510497 0.966227559207326 243.78570633053607
Source: 84.98389191804343 161.41346990697005 0.42687482279160693
Result: 84.98389191804343 161.41346990697005 0.42687482279160693 175.88636173387752
Source: 41.2157189833316 183.7805612769555 0.8526334612957889
Result: 41.2157189833316 183.7805612769555 0.8526334612957889 351.05462708670643
Source: 64.10045470658498 177.6245183820768 0.45865187310115585
Result: 64.10045470658498 177.6245183820768 0.45865187310115585 238.69817153143836
Source: 90.75887640065028 102.74910508662799 0.5587899438090776
Result: 90.75887640065028 102.74910508662799 0.5587899438090776 111.93792208610627
Source: 72.57091960652993 193.6281107452519 0.14774417368351056
Result: 72.57091960652993 193.6281107452519 0.14774417368351056 563.5424547948368
Source: 26.0141434383395 162.68706161680169 0.04636680164993484
Result: 26.0141434383395 162.68706161680169 0.04636680164993484 892.5615183143238
Source: 46.78279524338451 146.33711897576532 0.3917262008323986
Result: 46.78279524338451 146.33711897576532 0.3917262008323986 389.54342832538055
Source: 66.2432267618697 177.43117377602027 0.2806944762689644
Result: 66.2432267618697 177.43117377602027 0.2806944762689644 395.8580387855684
Source: 45.78538204456565 150.72235891535325 0.9258727585138075
Result: 45.78538204456565 150.72235891535325 0.9258727585138075 211.65347460163247
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\projects\Lab-6-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49780' '-XX:+ShowCodeDetail
sInExceptionMessages' '-cp' 'C:\Users\Borus\AppData\Roaming\Code\User\workspaceStorage\202763ceddce9ee57ffc339070f0064\redhat.java\jdt_ws\Lab-6-2025_4789df95\bin' 'Main'
Source: 45.78538204456565 150.72235891535325 0.9258727585138075
Result: 45.78538204456565 150.72235891535325 0.9258727585138075 211.65347460163247
Source: 24.25894920838152 158.94033632481197 0.7766282697727981
Result: 24.25894920838152 158.94033632481197 0.7766282697727981 1150.9474959148213
Source: 56.436659908285314 116.2608771023363 0.6705414567666743
Result: 56.436659908285314 116.2608771023363 0.6705414567666743 175.3739825599439
Source: 6.326179863091563 153.2293686897837 0.23279592734646182
Result: 6.326179863091563 153.2293686897837 0.23279592734646182 414.2164486438902
Source: 88.15588654871311 162.6091280000085 0.42090139133691507
Result: 88.15588654871311 162.6091280000085 0.42090139133691507 165.42951841414373
Source: 79.9887625306483 199.16045902157077 0.32415550898071784
Result: 79.9887625306483 199.16045902157077 0.32415550898071784 263.12363431211105
Source: 70.17253457331951 188.01660750941784 0.07041445132146107
Result: 70.17253457331951 188.01660750941784 0.07041445132146107 307.05764091368206
Source: 35.213101386000924 176.10791188319405 0.9671545659315761
Result: 35.213101386000924 176.10791188319405 0.9671545659315761 328.30682868760897
Source: 89.16218379990156 168.99559396580577 0.8094750594754875
Result: 89.16218379990156 168.99559396580577 0.8094750594754875 186.41652839521572
Source: 89.23903914355232 131.5117935848864 0.6246011559712774
Result: 89.23903914355232 131.5117935848864 0.6246011559712774 143.12410900825623
Source: 30.160018475877347 194.76261987829758 0.990615077433306
Result: 30.160018475877347 194.76261987829758 0.990615077433306 335.71210827373517
Source: 10.528605283493341 104.33612468827646 0.46655407670387494
Result: 10.528605283493341 104.33612468827646 0.46655407670387494 160.56989599533787
Source: 8.642516850087201 164.9672950647306 0.6016797773536543
Result: 8.642516850087201 164.9672950647306 0.6016797773536543 305.60695728209095
Source: 62.203319626348076 185.447254674185 0.4536365273332382
Result: 62.203319626348076 185.447254674185 0.4536365273332382 652.4898119592228
Source: 6.667990955325765 104.22411561708213 0.538381276963728
Result: 6.667990955325765 104.22411561708213 0.538381276963728 228.22718927308938
Source: 90.36062655502187 180.92799304605694 0.6591806105315687
Result: 90.36062655502187 180.92799304605694 0.6591806105315687 371.3854931285127
Source: 50.96658863875946 171.96286668339872 0.11763563008532985
Result: 50.96658863875946 171.96286668339872 0.11763563008532985 392.73743588221396
Source: 7.660107148483386 169.34319688333623 0.3966086549270931
Result: 7.660107148483386 169.34319688333623 0.3966086549270931 418.27123871351824
Source: 95.87038366095685 171.11325275662898 0.5230900515738698
Result: 95.87038366095685 171.11325275662898 0.5230900515738698 245.08873968240567
Source: 47.5701049129448 168.972367600239 0.9684577694432778
Result: 47.5701049129448 168.972367600239 0.9684577694432778 538.5642460539374
Source: 98.9741711160926 184.1393984739912 0.1242459936701253
Result: 98.9741711160926 184.1393984739912 0.1242459936701253 209.84322351846032
Source: 89.66536392381322 156.02492888673538 0.10345331679610814
Result: 89.66536392381322 156.02492888673538 0.10345331679610814 186.48222795583516
Source: 2.4176081501559588 189.9398022082718 0.8605850172567993
Result: 2.4176081501559588 189.9398022082718 0.8605850172567993 424.1407093362136
Source: 81.3663173063351 199.85379631713056 0.39803832338460865
Result: 81.3663173063351 199.85379631713056 0.39803832338460865 285.00176055241695
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\projects\Lab-6-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49780' '-XX:+ShowCodeDetail
s\InExceptionMessages' '-cp' 'C:\Users\Iborus\AppData\Roaming\Code\User\workspaceStorage\202763c4dccc9e57ffc339070f0064\redhat_java\jdt_ws\Lab-6-2025_4789df95\bin' 'Main'
Source: 81.3663173063351 199.85379631713056 0.39803832338460865
Result: 81.3663173063351 199.85379631713056 0.39803832338460865 285.00176055241695
Source: 57.679317974782194 140.1284145119498 0.13210306853262266
Result: 57.679317974782194 140.1284145119498 0.13210306853262266 315.0094716547095
Source: 6.510658394919478 106.18285305335266 0.328764579936373
Result: 6.510658394919478 106.18285305335266 0.328764579936373 170.94470904367867
Source: 56.77358001720294 198.21244083187963 0.8876598990304313
Result: 56.77358001720294 198.21244083187963 0.8876598990304313 389.70544566968374
Source: 76.14780791214552 184.36397249548622 0.6519517423381262
Result: 76.14780791214552 184.36397249548622 0.6519517423381262 655.9897521570193
Source: 57.74022345887546 170.18248169583916 0.8525495993197978
Result: 57.74022345887546 170.18248169583916 0.8525495993197978 405.6763939351926
Source: 41.94650578215051 179.67921533311036 0.25702080541127337
Result: 41.94650578215051 179.67921533311036 0.25702080541127337 282.7872057681442
Source: 17.318383511131895 180.03067907907658 0.08669853948577766
Result: 17.318383511131895 180.03067907907658 0.08669853948577766 420.7800755149324
Source: 57.608955075506316 136.70380338167104 0.7899081920072343
Result: 57.608955075506316 136.70380338167104 0.7899081920072343 477.2421873655648
Source: 68.69468776151854 168.7124381937366 0.3148970325017093
Result: 68.69468776151854 168.7124381937366 0.3148970325017093 268.90750127676233
Source: 38.673750425570965 184.74551339378158 0.6129665694121982
Result: 38.673750425570965 184.74551339378158 0.6129665694121982 362.0337831686481
--- Задание 3: simpleThreads() ---
Source: 60.4349415141733 169.7095716848761 0.786644604889118
Source: 62.14129602312855 125.61531907525003 0.5705545006268187
Source: 52.63684845734573 151.16959676658968 0.8883985024324049
Source: 44.97411576013336 154.0916123554508 0.3365606155323182
Source: 14.290109089159404 146.32401919495788 0.5410460040426929
Source: 97.19257050954234 127.05440806754105 0.3673394874068491
Source: 20.356930381697826 112.40739558615475 0.9024894188162496
Source: 69.03046489150753 117.74303994782686 0.394994174748617
Source: 35.14393039703393 193.90735542499624 0.01873309341854157
Source: 16.85989677069356 142.31593690507736 0.5390969317288915
Source: 76.64093650084195 185.41183046248662 0.7026378918866651
Source: 3.2996836385308814 148.82393991125832 0.445972357516647
Source: 64.99671292910172 119.46239783688554 0.8253345740757219
Source: 84.78107954382395 179.82256301172896 0.905092456616001
Source: 18.02295211178207 182.9293363360286 0.7117020091766979
Source: 61.96137356097763 124.63877645221908 0.7589896216232497
Source: 81.61253346113959 190.42404625511185 0.5813166442772235
Source: 93.37602970297624 102.30795734765425 0.17309406017506768
Source: 7.976476473448669 168.26445387435712 0.22627935715865588
Source: 97.71997192416604 141.1966417694975 0.5819017405498972
Source: 19.567207170065238 127.50900258199368 0.3673260493256344
Source: 95.14864364399479 146.32668716883023 0.877879174726095
Source: 21.60178651212583 129.574888174819 0.379628130373293
Source: 62.989460878978555 170.41646634585473 0.6595111600187321
va Ready
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\projects\Lab-6-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49780' '-XX:+ShowCodeDetail
s\InExceptionMessages' '-cp' 'C:\Users\Iborus\AppData\Roaming\Code\User\workspaceStorage\202763c4dccc9e57ffc339070f0064\redhat_java\jdt_ws\Lab-6-2025_4789df95\bin' 'Main'
Source: 52.63684845734573 151.16959676658968 0.8883985024324049
Source: 44.97411576013336 154.0916123554508 0.3365606155323182
Source: 14.290109089159404 146.32401919495788 0.5410460040426929
Source: 97.19257050954234 127.05440806754105 0.3673394874068491
Source: 20.356930381697826 112.40739558615475 0.9024894188162496
Source: 69.03046489150753 117.74303994782686 0.394994174748617
Source: 35.14393039703393 193.90735542499624 0.01873309341854157
Source: 16.85989677069356 142.31593690507736 0.5390969317288915
Source: 76.64093650084195 185.41183046248662 0.7026378918866651
Source: 3.2996836385308814 148.82393991125832 0.445972357516647
Source: 64.99671292910172 119.46239783688554 0.8253345740757219
Source: 84.78107954382395 179.82256301172896 0.905092456616001
Source: 18.02295211178207 182.9293363360286 0.7117020091766979
Source: 61.96137356097763 124.63877645221908 0.7589896216232497
Source: 81.61253346113959 190.42404625511185 0.5813166442772235
Source: 93.37602970297624 102.30795734765425 0.17309406017506768
Source: 7.976476473448669 168.26445387435712 0.22627935715865588
Source: 97.71997192416604 141.1966417694975 0.5819017405498972
Source: 19.567207170065238 127.50900258199368 0.3673260493256344
Source: 95.14864364399479 146.32668716883023 0.877879174726095
Source: 21.60178651212583 129.574888174819 0.379628130373293
Source: 62.989460878978555 170.41646634585473 0.6595111600187321
Source: 51.06188587585849 172.95613812659946 0.3296892720400223
Source: 6.25439583598482 129.65473988100186 0.6049300069561629
Source: 75.8057757186577 154.55172074080613 0.414745360890811
Source: 68.34366277496659 130.2228230229149 0.2873082514630334
Source: 76.50227836367667 184.157464588091533 0.9460861658676255
Source: 32.3330422480512 196.5806499008363 0.999931218451691
Source: 89.8172016388943 135.3229807228022 0.4679646379457902
Source: 88.2320464696476 117.53240011015852 0.40036919101797674
Source: 74.84350635215108 102.67975267689868 0.4512715914961013
Source: 66.68017986340275 151.08726863710757 0.058875233110448955
Source: 41.5040782083195 104.06591831501322 0.269213265043088
Source: 63.96207043302594 154.13105784620684 0.19740978013930655
Source: 63.70057083803694 148.67675128329574 0.923223560852293
Source: 10.25900701545025 127.9241517030937 0.21525534171310456
Source: 44.01610183361086 160.919359080774 0.707321353007394
Source: 53.944796638811624 195.44153750352416 0.799995791838818
Source: 19.22445669439954 190.6920486838862 0.5208074008426552
Source: 65.0101047259425 151.5705454129043 0.6861303620892924
Source: 17.39464945355062 144.20427440400008 0.4310100251414438
Source: 81.20929636151831 152.18178811684953 0.4469092799454283
Source: 5.743851495432628 140.85256936817654 0.43154635696863886
Source: 48.1809502825424 133.08048086894304 0.675740680894141
Source: 91.70840802437106 164.0682624279803 0.7439333322740124
Source: 28.180292426388633 174.91301605384445 0.723671382818966
Source: 65.1724987445342 125.45262872020139 0.855240269154133
Source: 79.40109102207673 154.07080097505518 0.7092461261561326
Source: 94.78416430170625 105.71485490513944 0.0803156354007073
Source: 28.722390651608725 151.4837520959147 0.327221252678746
Source: 16.18216681913067 180.52874047957972 0.81335867785543
Source: 90.8094650430636 193.266833947617 0.9525962132208682
Source: 75.09209517744249 147.62360054082706 0.15204061197410394
Source: 7.44977236887443 167.35106141352668 0.8657512371887378
Source: 13.14125169688915 125.2249366630709 0.9943900801674621
Source: 8.003608037724035 120.117918051181 0.66698340650438
Source: 9.297450605661457 139.360728309454 0.617821199190445
Source: 16.301504382386543 178.8103649911827 0.19369836554783726
Source: 14.61099029546138 154.54870955067227 0.30902718989583244
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\projects\Lab-6-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49788' '-Xc:ShowCodeDetail
sInExceptionHandlerMessages: '-cp 'C:\Users\Igorus\AppData\Roaming\Code\User\workspaceStorage\202763ceddce9e57ffc339070f0064\redhat.java\jdk_ws\Lab-6-2025_4789d4f95\bin\' 'Main'
Source: 84.80863181370 170.50803281013 0.32520489883388
Source: 84.2693499175306 119.5538992931106 0.86812092173461
Source: 81.6814989527926 162.797567344775 0.7910258312904703
Source: 84.70031156947367 189.9930834613444 0.5371088781608905
Source: 95.95841466731879 153.8146862652542 0.678833853405898
Source: 74.2598958247134 123.83736424867672 0.22384588927961336
Source: 69.28137152773502 129.47168045266144 0.3651782858640917
Source: 98.7218413105129 138.98326347388264 0.806313594855782
Source: 58.43994947022246 131.1373687080412 0.258455137378386
Source: 16.1117795369803 182.36972276079928 0.22517494421592177
Source: 18.137269197857794 106.2595166607281 0.8010642124679832
Source: 57.6116480543144 110.8755437116393 0.7898378134348279
Source: 63.7853392178895 188.2781688863867 0.64212695315843992
--- Загрузка 4: complicatedThreads() ---
Source: 37.8856211613863 140.55393788973913 0.6507715458015404
Result: 37.8856211613863 140.55393788973913 0.6507715458015404 + 211.13865138935272
Source: 12.88862684655525 188.62641287848101 0.5696049057822419
Result: 12.88862684655525 188.62641287848101 0.5696049057822419 + 456.253745062356
Source: 58.1121569899185 166.8755995110535 0.82803828596764143
Result: 58.1121569899185 166.8755995110535 0.82803828596764143 + 245.8904184670951
Source: 56.55238539988785 196.1319218991654 0.809902568472857442
Result: 56.55238539988785 196.1319218991654 0.809902568472857442 + 497.5997294818368
Source: 73.9804583538834 166.8439711518225 0.588401620640539
Result: 73.9804583538834 166.8439711518225 0.588401620640539 + 425.2077291299146
Source: 66.478240494931 195.29433860789379 0.430547029271311
Result: 66.478240494931 195.29433860789379 0.430547029271311 + 413.577219714331
Source: 24.968081195640778 147.2680112301445 0.004252365877183315
Result: 24.968081195640778 147.2680112301445 0.004252365877183315 + 488.5929113941769
Source: 1.802566769193129 174.5728245588804 0.564173318802353
Result: 1.802566769193129 174.5728245588804 0.564173318802353 + 1030.2238893517378
Source: 8.585832923730852 175.6516928327969 0.656920880818266
Result: 8.585832923730852 175.6516928327969 0.656920880818266 + 368.90790326537157
Source: 37.35817174918692 114.6158914443793 0.3329734145467808
Result: 37.35817174918692 114.6158914443793 0.3329734145467808 + 148.6586332500001
Source: 81.3057157518841 192.9609563141946 0.6770584807761922
Result: 81.3057157518841 192.9609563141946 0.6770584807761922 + 319.52761251052447
Source: 42.895282427823696 125.58551749610468 0.9414243767579965
Result: 42.895282427823696 125.58551749610468 0.9414243767579965 + 314.48919226724725
Source: 79.07298886932323 172.12948115522516 0.7446816818786227
Result: 79.07298886932323 172.12948115522516 0.7446816818786227 + 255.35802150450748
Source: 28.978812402404307 131.54138218071373 0.4238635445586710
Result: 28.978812402404307 131.54138218071373 0.4238635445586710 + 445.6989853774709
Source: 22.6858714575026 141.29834692207913 0.6396936244250795
Result: 22.6858714575026 141.29834692207913 0.6396936244250795 + 296.18584231654285
Source: 86.81664347942161 136.4892286364956 0.95939865633927
Result: 86.81664347942161 136.4892286364956 0.95939865633927 + 114.96083085349508
Source: 30.81548140743584 113.74384053013954 0.10644018677943971
Result: 30.81548140743584 113.74384053013954 0.10644018677943971 + 318.4381018595555
Source: 75.2882408408431 114.31172283755129 0.5915638567746805
Result: 75.2882408408431 114.31172283755129 0.5915638567746805 + 89.93138049378223
Source: 23.833744565929882 187.44885867699085 0.4896743334538038
Result: 23.833744565929882 187.44885867699085 0.4896743334538038 + 519.6951160277113
Source: 96.2638355302893 120.34280370600112 0.8429719314677822
Result: 96.2638355302893 120.34280370600112 0.8429719314677822 + 60.189199936767814
Source: 45.99278871686727 124.08479983184428 0.39863927948033584
Result: 45.99278871686727 124.08479983184428 0.39863927948033584 + 169.4906885170186
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\projects\Lab-6-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49788' '-Xc:ShowCodeDetail
sInExceptionHandlerMessages: '-cp 'C:\Users\Igorus\AppData\Roaming\Code\User\workspaceStorage\202763ceddce9e57ffc339070f0064\redhat.java\jdk_ws\Lab-6-2025_4789d4f95\bin\' 'Main'
Source: 27.67019543383716 114.633545428571 0.88755665647964937
Result: 27.67019543383716 114.633545428571 0.88755665647964937 + 636.1714774254943
Source: 84.0730092772534 175.29695217261896 0.7611245188814872
Result: 84.0730092772534 175.29695217261896 0.7611245188814872 + 235.93748470393984
Source: 4.24972815656489 123.2743808047738 0.86540486038451453
Result: 4.24972815656489 123.2743808047738 0.86540486038451453 + 289.204183965892
Source: 31.257385888644467 111.6295232349043 0.67924016913202582
Result: 31.257385888644467 111.6295232349043 0.67924016913202582 + 204.56769214707742
Source: 31.42993829236238 116.056168121253 0.897772113854882
Result: 31.42993829236238 116.056168121253 0.897772113854882 + 167.74485212421825
Source: 51.69117993622614 175.98883710557464 0.1104936149070479
Result: 51.69117993622614 175.98883710557464 0.1104936149070479 + 355.0108804814495
Source: 73.62170897572122 133.02891708481104 0.5678715414371105
Result: 73.62170897572122 133.02891708481104 0.5678715414371105 + 153.818854784077
Source: 2.9557656410357125 127.68371122921076 0.920459807326111
Result: 2.9557656410357125 127.68371122921076 0.920459807326111 + 248.9894780864375
Source: 54.37325498034296 105.8988928464631 0.8726888390038026
Result: 54.37325498034296 105.8988928464631 0.8726888390038026 + 148.35821911363857
Source: 48.8968868644249 155.80791275235424 0.5581323867264567
Result: 48.8968868644249 155.80791275235424 0.5581323867264567 + 737.68093945895992
Source: 63.0127446803979 142.26264422132556 0.4799675677139943
Result: 63.0127446803979 142.26264422132556 0.4799675677139943 + 231.09996186653495
Source: 38.781568297754866 131.47119527492388 0.14520124790028488
Result: 38.781568297754866 131.47119527492388 0.14520124790028488 + 277.4653991660182
Source: 88.57180358831168 101.5914787858721 0.802016919758889193
Result: 88.57180358831168 101.5914787858721 0.802016919758889193 + 60.56804219341922
Source: 5.69845056936043 116.38078928128854 0.18777169433226581
Result: 5.69845056936043 116.38078928128854 0.18777169433226581 + 239.4174304836183
Source: 22.244715896767443 197.96228830261026 0.028346577056468703
Result: 22.244715896767443 197.96228830261026 0.028346577056468703 + 487.34784648225684
Source: 48.5854262837832 174.36695861778998 0.5397431248428387
Result: 48.5854262837832 174.36695861778998 0.5397431248428387 + 314.72102125393354
Source: 98.5856707689444 189.2024400245037 0.9683288511683591
Result: 98.5856707689444 189.2024400245037 0.9683288511683591 + 197.14602122523283
Source: 23.979914775131702 159.68625363380194 0.4965662062114705
Result: 23.979914775131702 159.68625363380194 0.4965662062114705 + 268.13882688889215
Source: 69.3170342186563 137.22702305742183 0.287171988923818
Result: 69.3170342186563 137.22702305742183 0.287171988923818 + 145.954379485359
Source: 22.18034582722216 176.84809197067297 0.1483476182695914
Result: 22.18034582722216 176.84809197067297 0.1483476182695914 + 315.24153488215893
Source: 28.9883813178495 149.3441419443816 0.23566664885717548
Result: 28.9883813178495 149.3441419443816 0.23566664885717548 + 245.8801861165757
Source: 77.8672599410842 136.69428645615355 0.37119310703048
Result: 77.8672599410842 136.69428645615355 0.37119310703048 + 159.2418185571625
Source: 36.53300881900835 135.2858808289346 0.7567644831517892
Result: 36.53300881900835 135.2858808289346 0.7567644831517892 + 314.281291989239
Source: 95.154818245788074 154.653136896989 0.8857377278048415
Result: 95.154818245788074 154.653136896989 0.8857377278048415 + 270.687928747256
Source: 2.89845586294102 139.73175893163787 0.316622275980152
Result: 2.89845586294102 139.73175893163787 0.316622275980152 + 1585.2951287310793
Source: 16.70077791818575 163.65773148080756 0.6616430240549591
Result: 16.70077791818575 163.65773148080756 0.6616430240549591 + 333.27433016599807
Source: 32.5973353399852 135.1039542532836 0.6649080153617249
Result: 32.5973353399852 135.1039542532836 0.6649080153617249 + 304.81162221188566
PS C:\projects\Lab-6-2025>
```